

Task 1: Learning ROS2, TurtleBot3, SLAM and Nav2

Surendar

August 28, 2026

1 Introduction

This report summarizes what I learned while completing Task 1: setting up ROS2 Humble with Gazebo, understanding TurtleBot3, learning the basics of SLAM and Nav2, and finally running a simulated robot that builds a map and navigates to a goal point.

2 Environment Setup

I installed ROS2 Humble on Ubuntu 22.04 along with Gazebo Classic, which is the simulator compatible with this ROS2 distribution. I also set up a TurtleBot3 workspace (`turtlebot3_ws`) with the TurtleBot3 packages built from source, and set the `TURTLEBOT3_MODEL` environment variable (e.g. `burger` or `waffle_pi`) so the correct robot model gets spawned in simulation.

3 Learning TurtleBot3

TurtleBot3 is a small, ROS2-compatible differential-drive robot commonly used for learning navigation and SLAM. It comes in a few variants (Burger, Waffle, Waffle Pi) that differ in sensors and physical size. Key things I learned:

- The robot publishes odometry (`/odom`) and laser scan data (`/scan`) from its onboard LIDAR, and subscribes to `/cmd_vel` for velocity commands.
- `teleop_twist_keyboard` can be used to manually drive the robot using the keyboard, which is useful for testing before automating navigation.
- The robot model, sensors, and physical parameters (like wheel separation and diameter) are defined in an SDF/URDF file that gets loaded into Gazebo when the entity is spawned.

4 SLAM: Localization and Mapping

SLAM (Simultaneous Localization and Mapping) lets a robot build a map of an unknown environment while simultaneously tracking its own position within that map. I used the `slam_toolbox` package in its online asynchronous mode (`online_async_launch.py`), which:

- Subscribes to LIDAR scan data and odometry to incrementally build an occupancy grid map.
- Uses a solver plugin (Ceres Solver with the SCHUR-JACOBI preconditioner in this case) to optimize the pose graph as new scans come in.
- Publishes the map that can be visualized live in RViz.

Localization is the robot figuring out where it is on a map, while *mapping* is building that map in the first place – SLAM does both at once, which is why it is harder than doing either individually.

5 Frames and the TF Tree

ROS2 keeps track of the relationships between coordinate frames (e.g. `map`, `odom`, `base_link`, `base_scan`) using the `tf2` library. The command that visualizes the entire flow of frames and how they are connected is:

```
ros2 run tf2_tools view_frames
```

This generates a PDF showing the full TF tree, which is useful for debugging when a frame is missing or a transform is not being published correctly. For visualizing the node and topic connections (rather than frames), `rqt_graph` is the equivalent tool.

6 Nav2: Navigation

Nav2 is the ROS2 navigation stack that takes the map produced by SLAM and plans a path for the robot to reach a goal pose while avoiding obstacles. It brings together several components: a global planner (computes a path across the whole map), a local planner / controller (follows that path while avoiding obstacles in real time using the costmap), and behavior trees that coordinate recovery actions if the robot gets stuck. Once `nav2_bringup` is launched alongside the map, a goal pose can be sent through RViz using the “2D Goal Pose” tool, and the robot autonomously drives itself there.

7 Implementation

I chose the default TurtleBot3 world in Gazebo as my environment. The robot was spawned, driven manually to build up the map using `slam_toolbox`, and then Nav2 was used to send it to a goal point autonomously.

7.1 Spawning the Robot and Starting SLAM

Figure 1 shows the terminals used to spawn the TurtleBot3 (Burger model) into Gazebo, launch `gzclient` for visualization, start the `slam_toolbox` online async node, and run `teleop_twist_keyboard` to drive the robot around manually.

7.2 Building the Map

Figure 2 shows RViz and Gazebo side by side while the robot was driven around the environment. The occupancy grid map (grey/white/black) is being built live in RViz as the LIDAR scans (red points) are processed by `slam_toolbox`.

7.3 Navigating with Nav2

[Placeholder – to be replaced with a screenshot of Nav2 running: the completed map, the robot’s costmaps, and a goal pose being sent and followed in RViz.]

8 Single Launch File

To tie the whole task together, I wrote a single ROS2 launch file (`turtlebot3_slam_nav2_launch.py`) that starts Gazebo with the TurtleBot3 world, spawns the robot, starts `slam_toolbox`, brings up Nav2, and opens RViz – all with one command:

```
ros2 launch turtlebot3_slam_nav2_launch.py
```

The file and this report are both included in the submitted GitHub repository.

```

Activities  x-terminal-emulator  Fri Aug 28 20:35
surendar@surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx: ~
[robot_state_publisher-2] [INFO] [1787929421.727703715] [robot_state_publisher]: get segment wheel_right
[spawn_entity.py-3] [INFO] [1787929421.834744698] [spawn_entity]: Spawn Entity started
[spawn_entity.py-3] [INFO] [1787929421.934964285] [spawn_entity]: Loading entity XML from file /home/surendar/turtlebot3_ws/install/turtlebot3_gazebo/share/turtlebot3_gazebo/models/turtlebot3_burger/model.sdf
[spawn_entity.py-3] [INFO] [1787929421.935679786] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-3] [INFO] [1787929421.935843736] [spawn_entity]: Waiting for service /spawn_entity
[spawn_entity.py-3] [INFO] [1787929422.040072891] [spawn_entity]: Calling service /spawn_entity
[gzserver-1] [INFO] [1787929422.558674484] [turtlebot3_lm]: <initial_orientation_w_reference> is unset, using default value of false to comply with REP 145 (world as orientation reference)
[spawn_entity.py-3] [INFO] [1787929422.647487932] [spawn_entity]: Spawn status: SpawnEntity: Successfully spawned entity [Burger]
[gzserver-1] [INFO] [1787929422.758790522] [turtlebot3_diff_drive]: Wheel pair 1 separation set to [0.16 0.060]
[gzserver-1] [INFO] [1787929422.738835577] [turtlebot3_diff_drive]: Wheel pair 1 diameter set to [0.060 0.060]
[gzserver-1] [INFO] [1787929422.759459185] [turtlebot3_diff_drive]: Subscribed to [/cmd_vel]
[gzserver-1] [INFO] [1787929422.768778838] [turtlebot3_diff_drive]: Advertise odometry on /odom
[gzserver-1] [INFO] [1787929422.762189358] [turtlebot3_diff_drive]: Publishing odom transforms between [odom] and [base_footprint]
[gzserver-1] [INFO] [1787929422.768897629] [turtlebot3_joint_state]: Going to publish joint [wheel_left_joint]
[gzserver-1] [INFO] [1787929422.768131583] [turtlebot3_joint_state]: Going to publish joint [wheel_right_joint]
[INFO] [spawn_entity.py-3]: process has finished cleanly [pid 6215]

surendar@surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx: ~
surendar@surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx: ~$ ros2 launch slam_toolbox online_async.launch.py
[INFO] [launch]: All log files can be found below /home/surendar/.ros/log/2020-08-28-20:34:27-226205-surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx-6436
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [async_slam_toolbox_node-1]: process started with pid [6438]
[async_slam_toolbox_node-1] [INFO] [1787929467.294924886] [slam_toolbox]: Node using stack size 40000000
[async_slam_toolbox_node-1] [INFO] [1787929467.313853978] [slam_toolbox]: Using solver plugin solver_plus: CeresSolver
[async_slam_toolbox_node-1] [INFO] [1787929467.313986839] [slam_toolbox]: CeresSolver: Using SCHUR_JACOBI preconditioner
[async_slam_toolbox_node-1] [WARN] [1787929467.471423878] [slam_toolbox]: Minimum Laser range setting (0.0 m) exceeds the capabilities of the used Lidar (0.1 m)
[async_slam_toolbox_node-1] [WARN] [1787929467.471593171] [slam_toolbox]: maximum Laser range setting (2.0 m) exceeds the capabilities of the used Lidar (1.5 m)
[async_slam_toolbox_node-1] info: clipped range threshold to be within minimum and maximum range!
[async_slam_toolbox_node-1] Registering sensor: [Custom Described Lidar]

surendar@surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx: ~$ rviz2
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland to run on wayland anyway
[INFO] [1787929480.442779124] [rviz2]: Stereo is NOT SUPPORTED
[INFO] [1787929480.442967487] [rviz2]: OpenCL version: 4.6 (CLCL 4.6)
[INFO] [1787929480.438890943] [rviz2]: Stereo is NOT SUPPORTED
[INFO] [1787929517.338356804] [rviz2]: Trying to create a map of size 80 x 162 using 1 swatches
[ERROR] [1787929517.362506366] [rviz2]: Vertex Program: /usr/lib/llvm-8/clang/lib/Target/AArch64/AArch64Target.tbaa
/usr/lib/llvm-8/clang/lib/Target/AArch64/AArch64Target.tbaa: Image Prog: /usr/lib/llvm-8/clang/lib/Target/AArch64/AArch64Target.tbaa
Active swatches with a different type refer to the same texture image unit

surendar@surendar-OMEN-by-HP-Gaming-Laptop-16-xd0xxx: ~$ 105x12
w x d
a x s
x

w/x : Increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi : ~ 0.26)
a/d : Increase/decrease angular velocity (Burger : ~ 2.04, Waffle and Waffle Pi : ~ 1.02)
space key, s : force stop
CTRL-C to quit

```

Figure 1: Spawning TurtleBot3 in Gazebo and starting the SLAM and teleop nodes.

9 Conclusion

Through this task I learned how ROS2, Gazebo, SLAM, and Nav2 fit together: Gazebo simulates the robot and environment, `slam_toolbox` builds a map from LIDAR and odometry data, `tf2` keeps all the coordinate frames consistent, and Nav2 uses the resulting map to plan and execute navigation to a goal. Combining all of these into a single launch file gave me a clearer picture of how a real navigation stack is structured end to end.

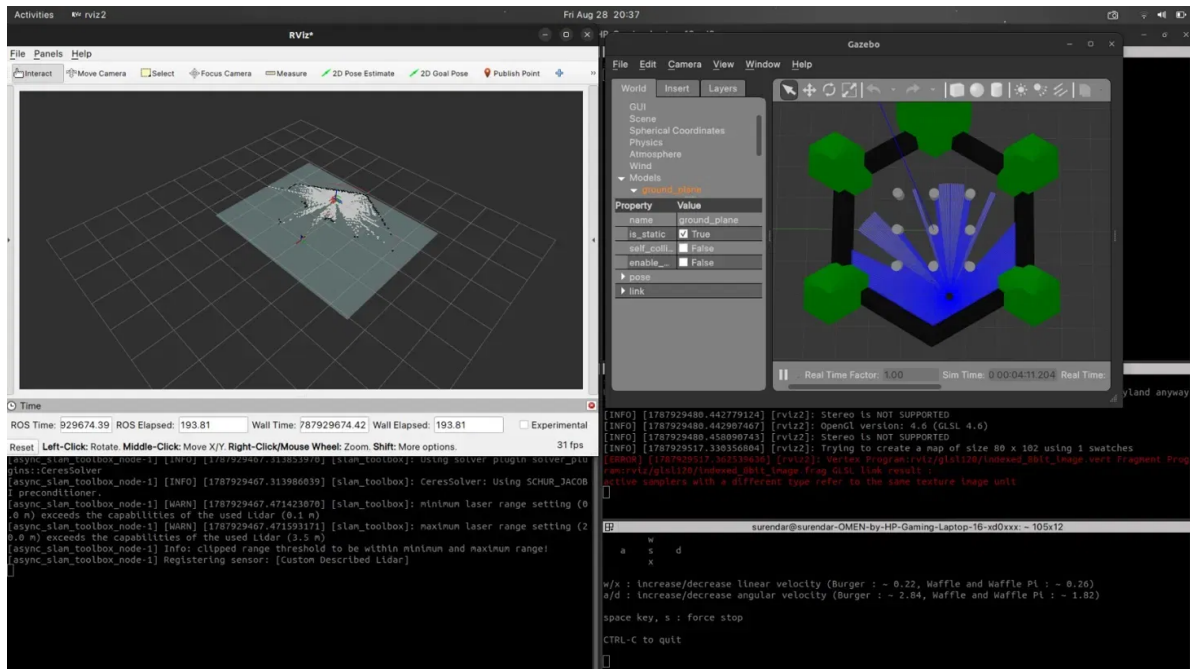


Figure 2: Live SLAM map being built in RViz alongside the Gazebo simulation.